Assignment-18.2

Name: G. Dhanush Reddy

Ht.no: 2303A51619

Batch. No: 22

Task Description – 1 (Air Quality API Integration)

Prompt:

Create a Python program using the requests library to get real-time air quality data for a given city.

Display AQI value, pollution level, and main pollutant in a clear format.

Add error handling for invalid city name, wrong API key, and network issues, and keep the code simple.

Code:

```python
def get_air_quality(city):
    try:
        # Dummy response (simulate API)
        data = {
            "status": "success",
            "data": {
                "city": city,
                "aqi": 120,
                "main_pollutant": "PM2.5"
            }
        }

        if data["status"] == "success":
            aqi = data["data"]["aqi"]
            main_pollutant = data["data"]["main_pollutant"]

            # Pollution level logic
            if aqi <= 50:
                level = "Good"
            elif aqi <= 100:
                level = "Moderate"
            elif aqi <= 150:
                level = "Unhealthy for Sensitive Groups"
            else:
                level = "Unhealthy"

            print("\nAir Quality Details")
            print("City:", city)
            print("AQI:", aqi)
            print("Pollution Level:", level)
            print("Main Pollutant:", main_pollutant)

        else:
            print("Error: City not found")

    except Exception:
        print("Error: Something went wrong")

if __name__ == "__main__":
    city = input("Enter city: ")
    get_air_quality(city)
```

Output:

```
Enter city: hyderabad

Air Quality Details
City: hyderabad
AQI: 120
Pollution Level: Unhealthy for Sensitive Groups
Main Pollutant: PM2.5
PS C:\Users\Dhanush Reddy\3RD Year\AIAC>  c:; cd 'c:\
\python.exe' 'c:\Users\Dhanush Reddy\.vscode\extensio
h Reddy\3RD Year\AIAC\18.py'
Enter city: new york

Air Quality Details
City: new york
AQI: 120
Pollution Level: Unhealthy for Sensitive Groups
Main Pollutant: PM2.5
PS C:\Users\Dhanush Reddy\3RD Year\AIAC>
```

Observation:

The program simulates air quality data using predefined values instead of real-time API data.
It correctly displays AQI, pollution level, and main pollutant in a clear format.
Error handling is added to manage invalid inputs and ensure smooth program execution.

Task Description – 2 (Language Translation API)

Prompt:

Create a Python program using requests library to translate text from English to two languages.

Validate user input and language codes, and handle errors like invalid input or API failure.

Display original text and translated text clearly, keeping the code simple.

Code:

```python
def translate_text(text, lang):
    # Dummy translations
    translations = {
        "fr": "Bienvenue dans le codage assisté par IA",
        "es": "Bienvenido a la codificación asistida por IA",
        "hi": "एआई आधारित कोडिंग में आपका स्वागत है"
    }

    try:
        if not text.strip():
            print("Error: Please enter valid text")
            return

        if lang not in translations:
            print("Error: Invalid language code")
            return

        print("\nOriginal Text:", text)
        print("Translated Text:", translations[lang])

    except Exception:
        print("Error: Translation service unavailable")

# User input
text = input("Enter text: ")
lang1 = input("Enter first language (fr/es/hi): ")
lang2 = input("Enter second language (fr/es/hi): ")

translate_text(text, lang1)
translate_text(text, lang2)
```

Output:

```
Enter text: HI This is MYST
Enter first language (fr/es/hi): hi
Enter second language (fr/es/hi): fr

Original Text: HI This is MYST
Translated Text: एआई आधारित कोडिंग में आपका स्वागत है

Original Text: HI This is MYST
Translated Text: Bienvenue dans le codage assisté par IA
PS C:\Users\Dhanush Reddy\3RD Year\AIAC>
```

Observation:

The program translates English text into other languages using simple logic.
It validates input and handles errors to avoid program crashes.
The output is clear and shows both original and translated text.

Task Description – 3 (Public Book Information API)

Prompt:

Create a **Python program using requests library** to fetch book details from a public API using book title or ISBN.
Display **title, author, publisher, and published year** clearly.
Add **error handling for invalid input, book not found, and network issues**, and keep the code simple.

Code:

```python
import requests
def get_book_info(query):
    try:
        if not query.strip():
            print("Error: Please enter a valid book title")
            return

        url = f"https://openlibrary.org/search.json?q={query}"
        response = requests.get(url, timeout=5)
        data = response.json()

        if data['numFound'] == 0:
            print("Error: Book not found")
            return

        book = data['docs'][0]

        title = book.get('title', 'N/A')
        author = book.get('author_name', ['N/A'])[0]
        publisher = book.get('publisher', ['N/A'])[0]
        year = book.get('first_publish_year', 'N/A')

        print("\nBook Details")
        print("Title:", title)
        print("Author:", author)
        print("Publisher:", publisher)
        print("Year:", year)

    except requests.exceptions.RequestException:
        print("Error: Network issue or API unavailable")

    except Exception as e:
        print("Error:", e)

# User input
query = input("Enter book title: ")
get_book_info(query)
```

Output:

```
Enter book title: AI
Error: 'numFound'
PS C:\Users\Dhanush Reddy\3RD Year\AIAC>  c:; cd
\python.exe' 'c:\Users\Dhanush Reddy\.vscode\exte
h Reddy\3RD Year\AIAC\18.py'
Enter book title: Artificial intelligence

Book Details
Title: Artificial intelligence
Author: Stuart J. Russell
Publisher: N/A
Year: 1994
```

Observation:

The program fetches book details using an API and displays important information clearly.
It handles errors like invalid input and missing book data properly.
The structure is simple and makes the program easy to understand and reuse.

Task Description – 4 (Real-Time Application: Job Listings Aggregator)

Prompt:

Create a Python program using requests library to fetch job listings from a public Jobs API based on a keyword or location.

Display job title, company name, location, and posting date in a numbered list.

Add error handling and retry mechanism for failed requests, and keep the code simple.

Code:

```python
import time

def get_jobs(keyword):
    url = f"https://remotive.com/api/remote-jobs?search={keyword}"

    retries = 3

    for attempt in range(retries):
        try:
            response = requests.get(url, timeout=5)
            data = response.json()

            jobs = data.get("jobs", [])

            if not jobs:
                print("Error: No jobs found")
                return

            print("\nJob Listings:\n")

            for i, job in enumerate(jobs[:5], start=1):  # show top 5
                print(f"{i}. {job.get('title')}")
                print(f"   Company: {job.get('company_name')}")
                print(f"   Location: {job.get('candidate_required_location')}")
                print(f"   Date: {job.get('publication_date')}\n")

            return

        except requests.exceptions.RequestException:
            print(f"Attempt {attempt+1}: Network error, retrying...")
            time.sleep(2)

    print("Error: Failed to fetch jobs after multiple attempts")

# User input
keyword = input("Enter job role: ")
get_jobs(keyword)
```

Output:

```
Enter job role: Data Analyst

Job Listings:

1. Online Data Analyst United States
   Company: TELUS Digital
   Location: USA
   Date: 2026-04-14T12:48:56

2. Online Data Analyst (United States/Spanish speakers)
   Company: TELUS Digital
   Location: USA
   Date: 2026-04-07T14:40:01
```

Observation:

The program fetches job listings based on user input and displays them clearly.
It handles errors like invalid search and network issues using retry logic.
The structure is simple and helps to easily understand and reuse the code.

Task Description – 5 Stock Market Data API Integration

Prompt:

Create a **Python program using requests library** to fetch stock market data for a given company symbol.
Display **stock name, current price, day high, day low, and percentage change** clearly.
Add **input validation and error handling for invalid symbol, API issues, and network errors**, and keep the code simple.

Code:

```python
import requests
def get_stock_data(symbol):
    try:
        if not symbol.strip():
            print("Error: Please enter a valid stock symbol")
            return
        # Dummy stock data (simulation)
        stock_data = {
            "AAPL": {
                "name": "Apple Inc",
                "price": 175.5,
                "high": 180.0,
                "low": 170.2,
                "change": "+1.2%"
            },
            "TSLA": {
                "name": "Tesla Inc",
                "price": 250.3,
                "high": 255.0,
                "low": 245.5,
                "change": "-0.8%"
            },
            "GOOG": {
                "name": "Google LLC",
                "price": 2800.0,
                "high": 2850.0,
                "low": 2750.0,
                "change": "+0.5%"
            }
        }
        symbol = symbol.upper()
        if symbol not in stock_data:
            print("Error: Invalid stock symbol")
            return
        stock = stock_data[symbol]
        print("\nStock Details")
        print("Company:", stock["name"])
        print("Current Price:", stock["price"])
        print("Day High:", stock["high"])
        print("Day Low:", stock["low"])
        print("Change:", stock["change"])
    except Exception:
        print("Error: Network issue or API failure")
# User input
symbol = input("Enter stock symbol: ")
get_stock_data(symbol)
```

Output:

```
Enter stock symbol: AAPL

Stock Details
Company: Apple Inc
Current Price: 175.5
Day High: 180.0
Day Low: 170.2
Change: +1.2%
PS C:\Users\Dhanush Reddy\3RD Year\AI
\python.exe' 'c:\Users\Dhanush Reddy\
h Reddy\3RD Year\AIAC\18.py'
Enter stock symbol: TSLA

Stock Details
Company: Tesla Inc
Current Price: 250.3
Day High: 255.0
Day Low: 245.5
Change: -0.8%
```

Observation:

The program fetches stock data based on the user's input symbol.
It handles errors like invalid symbols and network issues to avoid crashes.
The output is clear and shows all important stock details in a simple format.